backend guru

Building microservices

A working three-service store: catalog, orders and payments — built, run, and broken on purpose.

Emre Akis · 2 hours

Contents

01

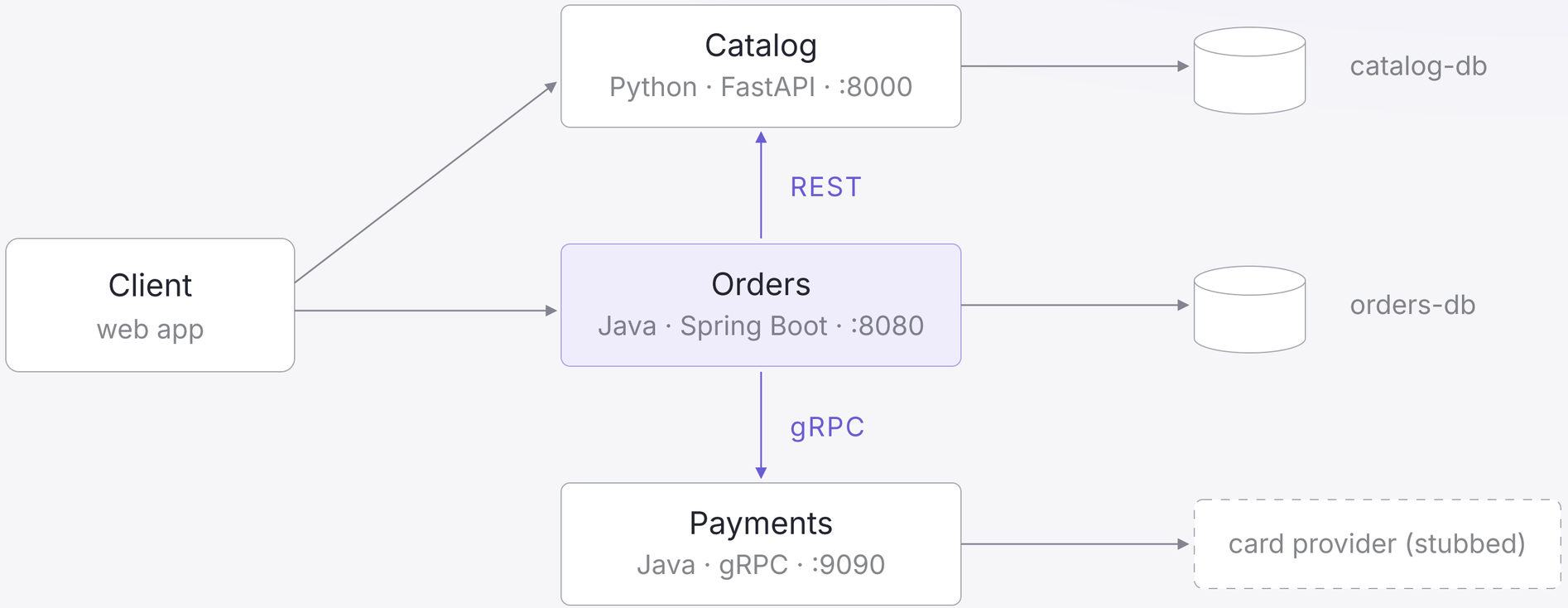
The system

backend guru

3

Three services — the smallest system that has every distributed problem worth learning: two protocols, a call chain, independent data, and a dependency that can fail mid-order.

The architecture



Service boundaries

One sentence

Each service's job fits in one sentence. Catalog: products and prices. Orders: checkout and order state. Payments: charges.

Own data

A service reads and writes only its own store. Anyone else's data arrives through that service's API — never its tables.

Own release

Each service builds, tests and deploys alone. If two services must always ship together, they are one service.

Database per service

What it buys

Schema changes stay local — catalog can migrate on a Tuesday without a meeting. No hidden coupling through shared tables, and each service picks the store that fits its shape.

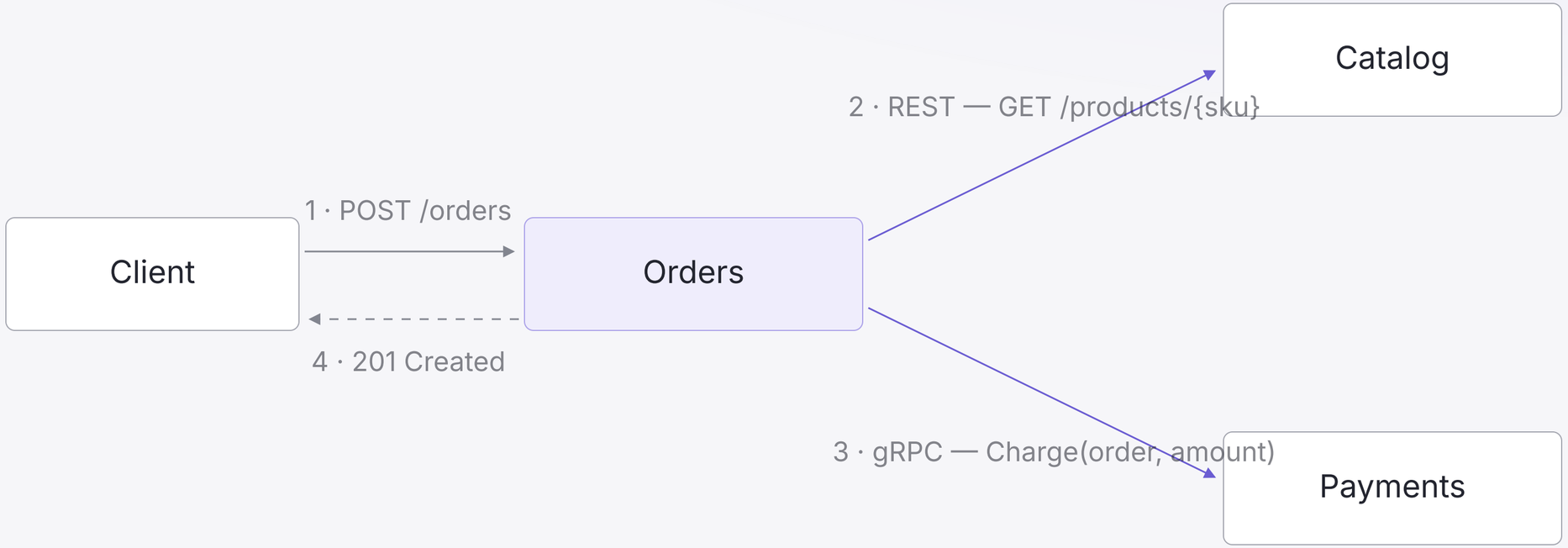
What it costs

No cross-service joins and no cross-service transactions. Data is duplicated by design — the order stores the price it was sold at — and consistency between services becomes eventual.

02

Service communication

Who calls whom



One user action, three services — the order is confirmed only after the price checks out and the charge is approved.

Catalog in FastAPI

```python
app = FastAPI(title="catalog")

@app.get("/products/{sku}")
def get_product(sku: str) -> Product:
    product = db.find(sku)
    if product is None:
        raise HTTPException(404, "unknown sku")
    return product

@app.get("/healthz")
def health() -> dict:
    return {"status": "up"}
```

The simplest service in the system — small enough to read in full, with a health check for the platform.

Orders in Spring

```
@Service
class OrderService {
  private final RestClient catalog;       // http://catalog:8000
  private final PaymentsClient payments; // payments:9090

  Order create(CreateOrder cmd) {
    var product = catalog.get()
        .uri("/products/{sku}", cmd.sku())
        .retrieve()
        .body(Product.class);              // 404 → OrderRejected

    var charge = payments.charge(cmd.orderId(),
                                 product.priceCents());
    return repo.save(Order.confirmed(cmd, charge));
  }
}
```

Payments over gRPC

PAYMENTS.PROTO — FROM SESSION 2

```
service Payments {
  rpc Charge(ChargeRequest)
      returns (ChargeReply);
}

message ChargeRequest {
  string order_id     = 1;
  int64  amount_cents = 2;
}
```

ORDERS — THE GENERATED CLIENT

```
var channel = ManagedChannelBuilder
    .forAddress("payments", 9090)
    .usePlaintext()
    .build();

var stub = PaymentsGrpc
    .newBlockingStub(channel)
    .withDeadlineAfter(2, SECONDS);

var reply = stub.charge(request);
```

Asynchronous events

- Not every consequence deserves a synchronous call in the checkout path
- Orders publishes `OrderConfirmed` once — email, analytics and inventory subscribe
- The order flow no longer waits on, or even knows about, its listeners
- The price: a broker to operate, and consistency that is eventual

03

Running it

One compose file

DOCKER-COMPOSE.YML

```yaml
services:
  catalog:                  # FastAPI
    build: ./catalog
    ports: ["8000:8000"]

  payments:                 # gRPC · Java
    build: ./payments
    ports: ["9090:9090"]
```

CONTINUED

```yaml
  orders:                   # Spring Boot
    build: ./orders
    ports: ["8080:8080"]
    environment:
      CATALOG_URL: http://catalog:8000
      PAYMENTS_HOST: payments:9090
    depends_on: [catalog, payments]

  orders-db:
    image: postgres:16
```

Configuration

Environment, not code

The image is identical everywhere; URLs, ports and secrets arrive as environment variables. Dev and prod differ by configuration only.

Names, not addresses

Services find each other by name — `http://catalog:8000`. Compose DNS resolves it today; Kubernetes or a mesh resolves it tomorrow, unchanged.

Observability

Logs

Structured, with the `order_id` on every line — one search reconstructs a checkout across all three services.

Metrics

Rate, errors and duration per endpoint. Alert on the caller's experience, not on CPU.

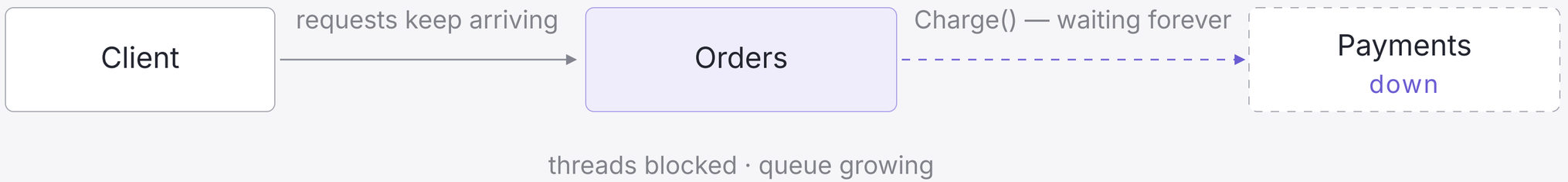
Traces

One trace id rides the request across the REST and gRPC hops — the waterfall shows exactly who was slow.

04

When things fail

A failure in the chain



Payments is down — but orders is what pages you. Without a deadline, every waiting thread is held hostage.

Timeouts, retries, breakers

Timeouts

Every remote call gets a deadline — two seconds for a charge. The default of forever is the worst possible value.

Retries

Retry only idempotent calls, with backoff and a budget. Unbounded retries against a struggling service finish it off.

Circuit breakers

After repeated failures, fail fast instead of waiting — return 503 with `Retry-After`, probe, and recover.

Exercise — break the system

- `docker compose stop payments` — then place an order
- Watch the orders logs: where exactly does the request die?
- Add a 2-second deadline to the gRPC stub and one retry with backoff
- Return 503 with `Retry-After` while payments is down
- 30 minutes — then we replay the failure together

github.com/emreakis/backend-bootcamp-store

backend guru

Thank you

Monolith trade-offs, contracts in REST and gRPC, and a three-service store that survives failure.

github.com/emreakis/backend-bootcamp-store

Emre Akis